

File: CircularQueue.cpp

Prompt: "How do I implement an enqueue for a circular buffer, and when the queue is full should I advance only tail or both head and tail?"

AI Output: Explained that enqueue should write to tail, then advance tail using modulo. If the queue is full, both head and tail should move so the oldest value is overwritten.

Modification: I used modulo arithmetic for both head and tail and made the enqueue keep only the newest N values for moving-average calculations.

File: CircularQueue.cpp

Prompt: "Should getAverage divide by capacity or by count in CircularQueue?"

AI Output: Explained that getAverage should divide by count because the queue may not be full yet.

Modification: I summed only the stored values and divided by count so unused array positions are not included.

File: CSVParser.cpp

Prompt: "How do I use ifstream and stringstream to read a Yahoo Finance CSV file line by line and split each row by commas?"

AI Output: Explained how to open the file with ifstream, read each line with getline, and use stringstream to separate fields by commas.

Modification: I used this structure in loadHistory and added validation for missing or malformed rows.

File: CSVParser.cpp

Prompt: "My CSVParser works for SPX and AMZN, but why does it read the wrong values for NVidia_stock_history.csv?"

AI Output: Explained that the NVDA file has a different column order, so fixed indexes like fields[4] for Close can be wrong.

Modification: I changed the parser to read the header row and dynamically find the indexes for Date, Open, High, Low, Close, and Volume.

File: CSVParser.cpp

Prompt: "Why can date strings in YYYY-MM-DD format be compared using normal string comparison, and how do I filter rows from 2000-01-01 to 2020-01-01?"

AI Output: Explained that lexicographic order matches chronological order because the year comes first, followed by month and day.

Modification: I used string comparison in dateInRange and checked the date before appending each row into PriceHistory.

File: CSVParser.cpp

Prompt: "What should CSVParser do if a row has missing values or cannot be converted with stod or stoll?"

AI Output: Explained that invalid rows should be skipped so the program does not crash while loading data.

Modification: I added checks for missing fields and used try-catch around numeric conversions.

File: FinancialAsset.cpp

Prompt: "What should go inside FinancialAsset.cpp if FinancialAsset is an abstract base class?"

AI Output: Explained that the constructor, destructor, getters, and setSector should be implemented, while pure virtual functions stay unimplemented.

Modification: I only implemented the shared non-virtual functions and left printSummary, calculateAnnualReturn, and getType for Stock and ETF.

File: FinancialAsset.cpp

Prompt: "Why does FinancialAsset need a virtual destructor?"

AI Output: Explained that a virtual destructor ensures derived objects such as Stock or ETF are destroyed correctly through a base-class pointer.

Modification: I kept the destructor virtual and implemented it even though it does not need special logic.

File: PriceHistory.cpp

Prompt: "Why do I still need PriceHistory.cpp if PriceNode.h already exists?"

AI Output: Explained that PriceNode only stores one node of data, while PriceHistory manages the full linked list, traversal, search, and deletion.

Modification: I implemented list operations in PriceHistory.cpp and did not create a separate PriceNode.cpp file.

File: PriceHistory.cpp

Prompt: "How do I append a new PriceNode to the end of a doubly linked list?"

AI Output: Explained the empty-list case and the normal case where tail->next points to the new node and newNode->prev points to the old tail.

Modification: I handled the empty-list case by setting both head and tail to the new node, then linked later nodes using next and prev.

File: PriceHistory.cpp

Prompt: "How do I delete every node in PriceHistory without causing a memory leak?"

AI Output: Explained that the destructor should loop from head to tail, save the next pointer, delete the current node, and continue.

Modification: I reset head, tail, and size after deleting all nodes.

File: PriceHistory.cpp

Prompt: "How do I implement findByDate and printRange in PriceHistory?"

AI Output: Explained that findByDate should linearly search the list and return nullptr if no date matches, while printRange should print only nodes between the start and end dates.

Modification: I returned nullptr for missing dates and formatted printRange with columns for Date, Open, High, Low, Close, and Volume.

File: PriceHistory.cpp

Prompt: "How do I implement custom forward and reverse iterators without using STL list?"

AI Output: Explained that the forward iterator should store a PriceNode pointer and move with next, while the reverse iterator should move with prev.

Modification: I implemented Iterator and Reverseliterator using raw PriceNode pointers and overloaded operator++, operator*, and operator!=.

File: Stock.cpp

Prompt: "How do I use CSVParser::loadHistory inside Stock::loadFromCSV?"

AI Output: Explained that loadFromCSV should call CSVParser::loadHistory, store the returned PriceHistory pointer, and return false if loading fails.

Modification: I deleted old history before loading new data to avoid memory leaks.

File: Stock.cpp

Prompt: "How do I search for a stock's closing price on a specific date?"

AI Output: Explained that Stock can call PriceHistory::findByDate and return the node's close value if found.

Modification: I returned -1.0 when the date was not found, matching the function requirement.

File: Stock.cpp

Prompt: "What is the difference between getYearStartPrice using Iterator and getYearEndPrice using Reverseliterator?"

AI Output: Explained that getYearStartPrice scans from oldest to newest to find the first trading day, while getYearEndPrice scans from newest to oldest to find the last trading day.

Modification: I used the forward iterator for start price and the reverse iterator for end price to meet the iterator requirement.

File: Stock.cpp

Prompt: "Why does annual return use the first and last trading days instead of January 1 and December 31 exactly?"

AI Output: Explained that the CSV skips weekends and market holidays, so those exact calendar dates may not exist.

Modification: I used getYearStartPrice and getYearEndPrice to find the first and last available trading days, then calculated annual return from those prices.

File: ETF.cpp

Prompt: "Since ETF inherits from Stock, do I need to rewrite loadFromCSV and PriceHistory logic in ETF.cpp?"

AI Output: Explained that ETF can reuse Stock's CSV loading and history functions and only needs to add expenseRatio, getType, printSummary, and CAGR logic.

Modification: I reused Stock behavior instead of duplicating code in ETF.

File: ETF.cpp

Prompt: "Why does ETF::~~ETF not need to delete history directly?"

AI Output: Explained that the ETF inherits from Stock, and the Stock destructor already deletes the owned PriceHistory pointer.

Modification: I left the ETF's destructor simple to avoid double-deleting the same history pointer.

File: ETF.cpp

Prompt: "How do I use pow() to calculate 10-year CAGR for an ETF?"

AI Output: Explained that CAGR is calculated with $\text{pow}(\text{endPrice} / \text{startPrice}, 1.0 / \text{years}) - 1$.

Modification: I included cmath, used the final available year from the tail node, subtracted 10 for the start year, and returned the result as a percentage.

File: TradeStack.cpp

Prompt: "How do I implement push and pop for a linked-list stack, and why is a stack useful for undoing trades?"

AI Output: Explained that push inserts at the top, pop removes the top node, and LIFO behavior is useful because undo should reverse the most recent trade first.

Modification: I updated the top pointer and size correctly, and used TradeStack pop() in undoLastTrade.

File: TradeStack.cpp

Prompt: "What should pop() return if TradeStack is empty?"

AI Output: Explained that the project assumes the stack should not be popped when empty, but checking the empty case is safer.

Modification: I checked isEmpty before popping in undoLastTrade and also handled the empty case inside pop.

File: OrderQueue.cpp

Prompt: "How do I implement enqueue and dequeue for a linked-list queue?"

AI Output: Explained that enqueue adds to the back and dequeue removes from the front.

Modification: I updated the front and back correctly for empty and non-empty queue cases.

File: OrderQueue.cpp

Prompt: "In a linked-list queue, what should happen to the front and back when the last order is dequeued?"

AI Output: Explained that after removing the last node, both front and back should become nullptr.

Modification: I updated dequeue so the queue state remains correct when it becomes empty.

File: OrderQueue.cpp

Prompt: "What should peek() do differently from dequeue() in OrderQueue?"

AI Output: Explained that peek returns the front order without removing it, while dequeue removes and returns the front order.

Modification: I implemented peek without changing front, back, or size.

File: Portfolio.cpp

Prompt: "How do I update cash balance when buying and selling shares?"

AI Output: Explained that buying subtracts shares times price from cash, while selling adds shares times price to cash.

Modification: I checked for enough cash before buying and enough shares before selling.

File: Portfolio.cpp

Prompt: "When buying more shares of a stock I already own, how should I update the average cost basis?"

AI Output: Explained that the new average should be based on old total cost plus new purchase cost divided by total shares.

Modification: I used a weighted average cost basis instead of replacing the old cost with the latest price.

File: Portfolio.cpp

Prompt: "Why should undoLastTrade not call buyShares or sellShares directly?"

AI Output: Explained that calling buyShares or sellShares would push a new trade record onto the stack, making undo create another trade.

Modification: I reversed the cash and holdings manually inside undoLastTrade instead.

File: Portfolio.cpp

Prompt: "What should happen when a sell order reduces a position's shares to zero?"

AI Output: Explained that the position should be removed from the holdings vector when no shares remain.

Modification: I erased the Position from the vector when the share count became zero.

File: Portfolio.cpp

Prompt: "How do I execute the market and limit orders from OrderQueue?"

AI Output: Explained that market orders execute immediately, limit buys execute at or below target price, and limit sells execute at or above target price.

Modification: I checked the order type and side before calling buyShares or sellShares, and skipped limit orders when the condition was not met.

File: Portfolio.cpp

Prompt: "How do I calculate total unrealized return for all portfolio positions?"

AI Output: Explained that total unrealized return should compare total market value against total cost basis across all holdings.

Modification: I summed shares times current price and shares times average cost basis before calculating the percentage return.

File: StockBST.cpp

Prompt: "How do I insert a node into a BST recursively, and where should duplicate return values go?"

AI Output: Explained that insert should compare the key with the current node and recurse left or right until a null spot is found. Duplicate values can consistently go to the right subtree.

Modification: I inserted smaller returns on the left and greater or equal returns on the right.

File: StockBST.cpp

Prompt: "How do I write inorder, preorder, and postorder traversal for StockBST?"

AI Output: Explained the three recursive traversal orders and that inorder prints the BST keys in sorted order.

Modification: I used inorder for sorted return output and kept preorder and postorder for the required BST display menu.

File: StockBST.cpp

Prompt: "How do I implement BST range search to find returns between two values?"

AI Output: Explained how to recursively search only branches that could contain values inside the low and high range.

Modification: I stored matching nodes in a vector so the menu could print all matching results.

File: StockBST.cpp

Prompt: "How do I search for a key and find the maximum key in a BST?"

AI Output: Explained that search compares the target key and moves left or right, while findMax follows right pointers until no right child exists.

Modification: I returned nullptr when search failed and used right-child traversal for findMax.

File: StockBST.cpp

Prompt: "What should StockBST::getHeight return for an empty tree, and how should clear delete the whole tree?"

AI Output: Explained that an empty tree can return -1 so a single-node tree has height 0, and clear should use postorder deletion.

Modification: I used -1 as the height base case and reset root to nullptr after clearing.

File: StockManager.h

Prompt: "Why does StockManager need to be implemented in the header file instead of a .cpp file?"

AI Output: Explained that template classes must be fully defined in the header so the compiler can generate code for StockManager<Stock> and StockManager<ETF>.

Modification: I implemented the constructor, destructor, add, remove, find, sort, and print functions directly in StockManager.h.

File: TradingStrategy.cpp

Prompt: "Why does TradingStrategy.cpp only implement printResult, calculateCAGR, and calculateMaxDrawdown?"

AI Output: Explained that backtest and getName are pure virtual, so each derived strategy implements its own version.

Modification: I kept the base class for shared calculations only and implemented each strategy's logic in its own .cpp file.

File: TradingStrategy.cpp

Prompt: "How do I calculate CAGR in a reusable strategy base class?"

AI Output: Explained the CAGR formula and why invalid inputs should return 0.0.

Modification: I implemented calculateCAGR once in TradingStrategy so all strategies can reuse it.

File: TradingStrategy.cpp

Prompt: "How do I calculate maximum drawdown from a list of portfolio values?"

AI Output: Explained that the algorithm should track the highest value so far and compute the largest percentage drop from that peak.

Modification: I stored the largest drawdown as a positive percentage.